

KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade



ResumeIQ project – AI-Powered Resume & Job Description Matcher

Project ID: PRJ003

Team ID: SPIRIT_QUINT (T003)

Team GitHub Repository: https://github.com/Geethika38830/AI-Resume-JD-Matcher

Student Name	Registration Number	Individual GitHub Link
N. Geethika	99230040381	https://github.com/Geethika38830/GEN_AI_MASTERY_PRACTICE_FILES.git
P. Sai Jagruthi	99230040402	https://github.com/jagruthi2005/Gen-AI-Class-Files.git
M. Hema Latha	99230040364	https://github.com/992300403644-del/Gen-Ai-Practice-codes
D. Gopi	99230041179	https://github.com/Gopi07-max/Gen-AI-Mastery.git
E. Rama Tulasi	99230040298	https://github.com/edulatulasi/Gen_Ai.git

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

Table of Contents

1. Introduction ..... Section 1
2. Problem Statement & Objectives ..... Section 2
3. System Architecture (RAG Pipeline) ..... Section 3
4. Implementation Details & Tech Stack ..... Section 4
5. Results and Testing ..... Section 5
6. Conclusion & Future Scope ..... Section 6

1 Introduction

ResumeIQ is an AI-powered full-stack web application designed to help job seekers analyze their resumes against specific job descriptions and understand their suitability for targeted job roles.

The application allows users to upload their resumes in PDF format, create job descriptions, and analyze the compatibility between their qualifications and industry requirements. It extracts relevant information from resumes, identifies matching skills, detects missing skills, and calculates a resume-job compatibility score.

By leveraging Artificial Intelligence, Natural Language Processing (NLP), semantic similarity, and machine learning techniques, ResumeIQ provides meaningful insights into candidates' strengths and areas that require improvement. The platform also generates personalized recommendations to help users improve their technical skills and enhance their job readiness.

The system is developed using React, TypeScript, Material UI, Python, FastAPI, and PostgreSQL, with AI-based matching capabilities and an optional Retrieval-Augmented Generation (RAG) pipeline.

The primary objective of ResumeIQ is to bridge the gap between candidate skills and job market requirements by providing intelligent resume analysis, skill-gap identification, and actionable career improvement recommendations through a unified platform.

2 Problem Statement & Objectives

2.1 Problem Statement

In today's competitive job market, candidates often face difficulties in identifying whether their resumes meet the requirements of specific job roles. Manually comparing resumes with job descriptions is time-consuming, inconsistent, and may result in overlooking important technical skills and qualifications.

Many job seekers are unaware of the skills they already possess, the skills required by employers, and the specific areas they need to improve to become suitable candidates

for their desired positions. Traditional resume evaluation methods do not always provide detailed skill-gap analysis or personalized recommendations for career improvement.

Therefore, there is a need for an intelligent system that can automatically analyze resumes against job descriptions, evaluate candidate suitability, identify matching and missing skills, and provide actionable recommendations to improve employability.

ResumeIQ addresses this problem by providing an AI-powered platform that automates resume-job matching and helps candidates understand and improve their job readiness.

2.2 Objectives

The primary objectives of the ResumeIQ application are:

1. Automated Resume Analysis: To develop a system that allows users to upload PDF resumes and automatically extract relevant information.

2. Job Description Management: To provide functionality for creating and managing job descriptions for different job roles.

3. Resume-Job Matching: To analyze uploaded resumes against job descriptions and calculate compatibility scores using AI-based matching techniques.

4. Skill Identification: To identify and display the skills that match the requirements of a particular job description.

5. Skill Gap Analysis: To detect missing or insufficiently represented skills by comparing candidate profiles with job requirements.

6. Personalized Recommendations: To generate suggestions that help candidates improve their technical knowledge and address identified skill gaps.

7. User Authentication: To implement secure user registration and login functionality using JWT-based authentication.

8. Match History Management: To maintain previous matching results so users can review their resume analysis over time.

9. Career Readiness Improvement: To help job seekers understand their strengths, identify improvement areas, and prepare more effectively for targeted job opportunities.

2.3 Expected Outcome

The expected outcome is a centralized AI-powered resume analysis platform that provides compatibility scores, matched skills, missing skills, and personalized recommendations, enabling candidates to make informed decisions about their career development and improve their readiness for employment.

3 System Architecture (RAG Pipeline)

3.1 System Architecture Overview

ResumeIQ follows a modular full-stack architecture that integrates frontend technologies, backend services, database management, and AI-powered resume analysis.

The system uses a Retrieval-Augmented Generation (RAG) pipeline as an optional component to enhance resume and job description analysis by retrieving relevant information and generating contextual explanations.

The architecture consists of the following major components:

1. **Frontend Layer:** React, TypeScript, Vite, and Material UI provide the user interface for registration, login, resume uploads, job description creation, and viewing matching results.
2. **Backend Layer:** FastAPI handles API requests, authentication, resume processing, job description management, and matching operations.
3. **Database Layer:** PostgreSQL stores user information, uploaded resume details, job descriptions, skills, and matching history.
4. **Resume Processing Layer:** PyMuPDF extracts textual content from uploaded PDF resumes for further analysis.
5. **Embedding Generation:** Sentence Transformers (all-MiniLM-L6-v2) converts resume and job-related text into numerical vector representations.
6. **Vector Database:** ChromaDB stores and retrieves vector embeddings for semantic similarity search.
7. **LLM Integration:** Ollama with Gemma 3 (4B) generates contextual explanations and recommendations using retrieved information.

3.2 RAG Pipeline Architecture

The RAG pipeline consists of two major phases: Document Ingestion and Query Processing.

A. Document Ingestion Phase

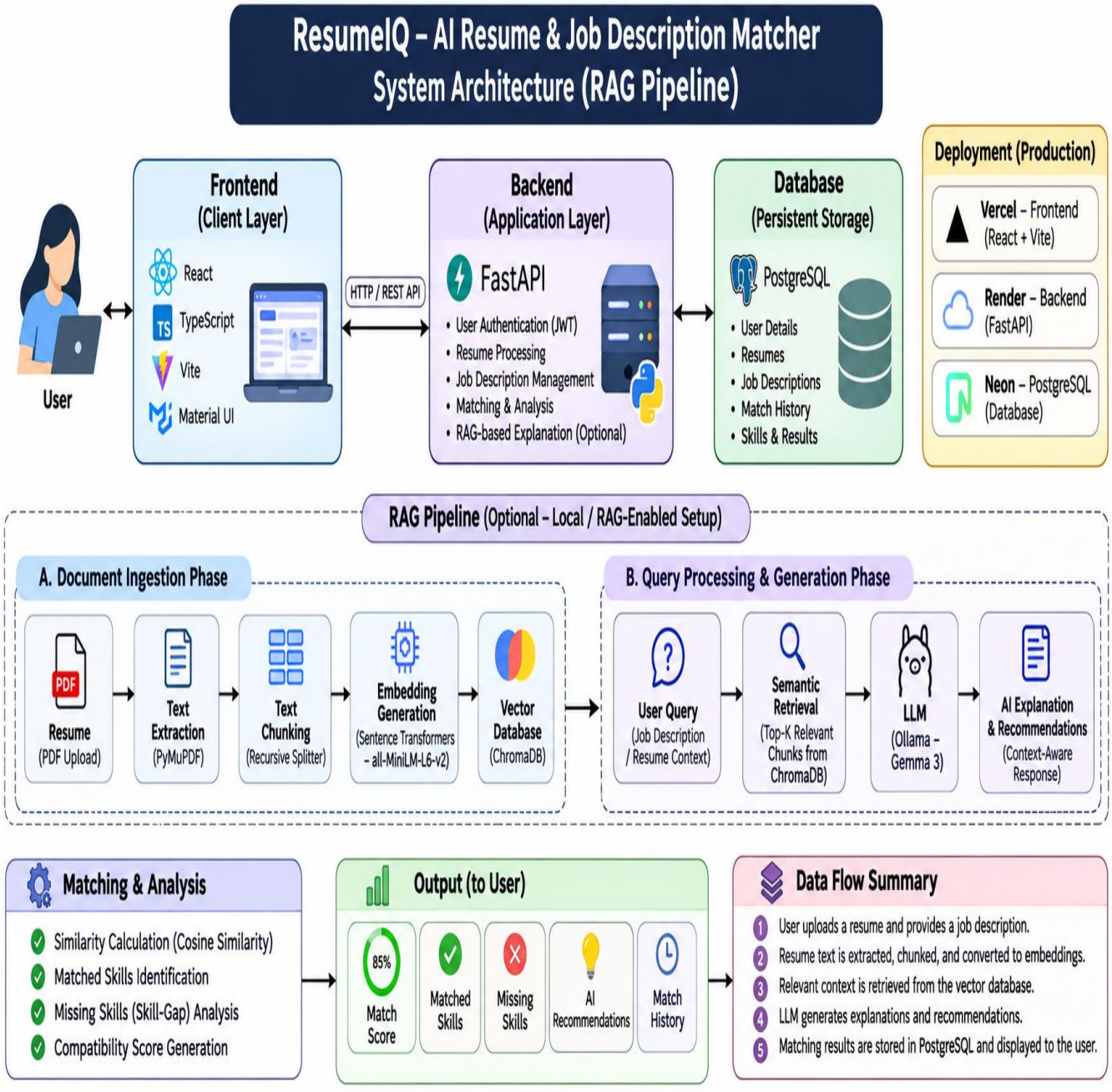
1. The user uploads a resume in PDF format.
2. PyMuPDF extracts the textual content from the uploaded document.
3. The extracted text is processed and divided into smaller meaningful chunks.
4. Sentence Transformers converts the text chunks into numerical embeddings.
5. The generated embeddings are stored in ChromaDB for efficient semantic retrieval.

B. Query Processing and Generation Phase

1. The user creates or selects a job description.
2. The resume and job description are processed by the matching service.
3. The relevant text is converted into embeddings.
4. ChromaDB retrieves relevant information using semantic similarity search.

5. The retrieved context is passed to the Gemma 3 language model through Ollama.
6. The language model generates contextual explanations and recommendations.
7. The application presents the matching score, matched skills, missing skills, and analysis results to the user.

3.3 System Architecture Diagram



3.4 Data Flow Architecture

The overall data flow follows these steps:

- Input: Users provide resumes and job descriptions through the frontend.
- Processing: The backend extracts and processes resume content and job

requirements.

- Analysis: Matching algorithms compare candidate skills with job requirements.
- Retrieval: The optional RAG component retrieves relevant contextual information from ChromaDB.
- Generation: The LLM generates explanations and recommendations.
- Storage: User details, resumes, job descriptions, and matching records are stored in PostgreSQL.
- Output: The frontend displays compatibility scores, matched skills, missing skills, and recommendations.

3.5 Deployment Architecture

The application is deployed using three separate services:

Component	Technology	Platform
Frontend	React + Vite	Vercel
Backend	FastAPI	Render
Database	PostgreSQL	Neon

4 Implementation Details & Tech Stack

4.1 Implementation Details

ResumeIQ is implemented as a full-stack web application that integrates frontend development, backend processing, database management, and AI-based resume analysis. The system is divided into multiple modules to ensure efficient processing, maintainability, and scalability.

1. Frontend Development
The frontend is developed using React, TypeScript, Vite, and Material UI. It provides an interactive interface through which users can register, log in, upload resumes, create job descriptions, view matching results, and manage their career profiles.

2. Backend Development
The backend is developed using Python and FastAPI. It manages API requests, user authentication, resume processing, job description management, matching operations, and communication with the database.

3. Resume Processing
The system accepts resumes in PDF format and uses PyMuPDF to extract textual information. The extracted content is processed and prepared for comparison with job descriptions.

4. Job Description Processing
Users can create and manage job descriptions containing job titles, technical skills, qualifications, and responsibilities. These requirements are used as input for resume matching and skill-gap analysis.

5. AI-Based Resume Matching
The matching service evaluates resume content against job requirements using text similarity and skill-matching techniques. It generates compatibility scores and identifies matched and missing skills.

6. Semantic Analysis and RAG Integration
The optional RAG pipeline uses Sentence Transformers to generate embeddings, ChromaDB to store and retrieve relevant information, and Ollama with Gemma 3 to generate contextual explanations and recommendations.

7. Database Management
PostgreSQL is used to store user accounts, resume information, job descriptions, skills, and matching records. SQLAlchemy provides database interaction through ORM models.

8. Authentication and Security
JWT-based authentication is implemented to manage user sessions and protect application resources. Environment variables are used to configure sensitive credentials and database connections.

4.2 Technology Stack

The following technologies are used in the development of ResumeIQ.

Component	Technology	Purpose
Programming Language	Python	Backend logic and AI processing
Frontend Framework	React	Building interactive user interfaces
Frontend Language	TypeScript	Type-safe frontend development
Build Tool	Vite	Frontend development and production builds
UI Framework	Material UI (MUI)	Responsive user interface components

Backend Framework	FastAPI	REST API development
ORM	SQLAlchemy	Database operations and object-relational mapping
Database	PostgreSQL	Persistent application data storage
Database Hosting	Neon	Managed PostgreSQL hosting
PDF Processing	PyMuPDF	Extracting text from PDF resumes
Embedding Model	Sentence Transformers	Generating semantic text embeddings
Embedding Model Name	all-MiniLM-L6-v2	Converting text into vector representations
Similarity Technique	Cosine Similarity	Measuring semantic similarity
Vector Database	ChromaDB	Storing and retrieving embeddings
LLM	Gemma 3 (4B)	Generating AI explanations in local RAG mode
LLM Runtime	Ollama	Running the local language model
Authentication	JWT	Secure user authentication
Frontend Deployment	Vercel	Hosting the React application
Backend Deployment	Render	Hosting the FastAPI application
Version Control	Git and GitHub	Source code management

4.3 Backend Modules

The backend is organized into the following major modules:

Module	**Description**
Authentication	User registration, login, and JWT authentication
Resume Management	PDF upload and resume text extraction

Job Description	Creating and managing job descriptions
Matching Service	Resume-job compatibility analysis
Skill Analysis	Identifying matched and missing skills
Dashboard	Providing user-specific application information
RAG Pipeline	Optional semantic retrieval and AI-generated explanations

4.4 Database Implementation

ResumeIQ uses PostgreSQL as its primary relational database, with SQLAlchemy for database operations.
The major database entities include:

- User: Stores user account and authentication information.
- Resume: Stores resume metadata and extracted content.
- JobDescription: Stores job titles and job requirements.
- Skill: Maintains skill-related information.
- Match: Stores resume-job matching results and analysis.

The database is hosted on Neon PostgreSQL in the production environment.

4.5 AI and RAG Implementation

The optional RAG-enabled workflow consists of the following technologies:

Component	**Technology**	**Function**
Text Extraction	PyMuPDF	Extracts resume text
Text Embeddings	Sentence Transformers	Generates vector representations
Embedding Model	all-MiniLM-L6-v2	Creates semantic embeddings

Vector Storage	ChromaDB	Stores document embeddings
Semantic Retrieval	Similarity Search	Retrieves relevant information
Language Model	Gemma 3 (4B)	Generates contextual explanations
Model Runtime	Ollama	Executes the local language model

The RAG pipeline enhances the matching workflow by retrieving relevant contextual information before generating explanations.
Production configuration note: The hosted Render backend currently runs with RAG_ENABLED=false to stay within the free instance's memory limit. The RAG workflow described above is supported in the local RAG-enabled configuration.

4.6 Production Deployment

The completed application is deployed using three hosting services:

1. Vercel: Hosts the React frontend and provides public access to the application.
2. Render: Hosts the FastAPI backend and processes API requests.
3. Neon: Provides managed PostgreSQL database storage.

Live Application: https://ai-resume-jd-matcher-pearl.vercel.app/

GitHub Repository: https://github.com/Geethika38830/AI-Resume-JD-Matcher

4.7 Summary

The integration of React, FastAPI, PostgreSQL, and AI-based matching technologies enables ResumeIQ to provide an end-to-end resume evaluation platform. Its modular architecture supports resume processing, job description analysis, compatibility scoring, skill-gap identification, and personalized recommendations. The optional RAG pipeline further extends the system with semantic retrieval and contextual AI-generated explanations.

5 Results and Testing

5.1 Results

The ResumeIQ application was successfully developed and deployed as a full-stack web application. It provides an integrated platform for resume uploading, job description management, resume-job compatibility analysis, skill-gap identification, and career profile management.
The application was tested using different job descriptions to evaluate how effectively it analyzes candidate resumes against various job requirements.

The major results obtained from the application include:
1. Successful Resume Upload: Users can upload PDF resumes, and the system extracts the textual information for further processing.
2. Job Description Creation: Users can create and save multiple job descriptions for different job roles.
3. Resume-Job Compatibility Analysis: The matching service compares resume content with job requirements and generates compatibility scores.
4. Matched Skills Identification: The system identifies skills present in the resume that match the selected job description.
5. Missing Skills Identification: The application highlights skills that are required by the job description but are not sufficiently represented in the resume.
6. AI-Based Recommendations: The application provides explanations and recommendations based on the matching analysis.
7. Match History: Previous matching results can be accessed through the application.
8. User Profile Management: Users can manage their account information and career preferences.
9. Successful Cloud Deployment: The frontend, backend, and database were deployed using Vercel, Render, and Neon PostgreSQL respectively.

5.2 Testing

Testing was performed to verify the functionality of individual modules and the integration of the complete application.
The following test cases were considered:

Test Case	Expected Result	Status
User Registration	New user account should be created	Passed
User Login	User should be authenticated successfully	Passed
PDF Resume Upload	Resume should be uploaded and processed	Passed
Job Description Creation	Job description should be saved	Passed

Resume Matching	Compatibility score should be generated	Passed
Matched Skills Analysis	Matching skills should be displayed	Passed
Missing Skills Analysis	Missing skills should be identified	Passed
Recommendations	Analysis-based recommendations should be displayed	Passed
Match History	Previous results should be accessible	Passed
Profile Management	Career profile should be saved	Passed
Database Persistence	Stored information should remain available after refresh	Passed
Frontend Deployment	Application should load through the Vercel URL	Passed
Backend Connectivity	Frontend should communicate with the Render API	Passed

Test statuses reflect the functionality you reported testing. They are not independent automated test results.

5.3 Test Scenarios

Scenario 1: AI/ML Engineer Job Matching

- Uploaded a candidate resume.
- Created an AI/ML Engineer job description.
- Executed the resume matching process.
- Observed the compatibility score and skill analysis.

Result: The application generated matching results and highlighted relevant skills and improvement areas.

Scenario 2: Full Stack Developer Job Matching

- Used the same candidate resume.
- Created a Full Stack Developer job description.
- Executed the matching process again.
- Compared the resulting skill analysis.

Result: The application generated a separate matching analysis for the Full Stack Developer role.

Scenario 3: Data Persistence

- Created user information and job descriptions.
- Uploaded a resume and generated matching results.
- Refreshed the application.
- Checked the stored information and history.

Result: Previously stored application information remained accessible after refreshing.

5.4 Deployment Results

The application was successfully deployed using cloud-based hosting services.

Component	Deployment Platform	Result
Frontend	Vercel	Successfully deployed
Backend	Render	Successfully deployed
Database	Neon PostgreSQL	Connected successfully
API Documentation	FastAPI Swagger	Accessible
Production Application	ResumeIQ	Accessible online

Live Application: https://ai-resume-jd-matcher-pearl.vercel.app/

GitHub Repository: https://github.com/Geethika38830/AI-Resume-JD-Matcher

5.5 Performance and Limitations

The application successfully performs its core resume analysis and matching operations. However, certain deployment considerations remain:

- The free Render backend may experience cold starts after periods of inactivity.
- The production environment currently uses RAG_ENABLED=false to stay within the free instance's memory limits.
- The full RAG workflow using Sentence Transformers, ChromaDB, and Ollama/Gemma is available in the local RAG-enabled configuration.
- Permanent storage of uploaded PDF files requires persistent or external file storage.

No numerical accuracy percentage or response-time benchmark is claimed because a formal performance evaluation has not yet been conducted.

5.6 Overall Testing Conclusion

The testing process confirms that ResumeIQ successfully performs its major functional operations, including user authentication, resume uploading, job description creation, compatibility analysis, skill-gap identification, and match history management.

The successful integration of the frontend, backend, and PostgreSQL database demonstrates that the application works as a complete deployed platform.

6 Conclusion & Future Scope

6.1 Conclusion

ResumeIQ is an AI-powered resume and job description matching platform developed to help job seekers evaluate their qualifications against industry job requirements. The application provides an integrated solution for resume analysis, job description management, compatibility scoring, skill-gap identification, and personalized career recommendations.

The system successfully integrates modern web technologies such as React, TypeScript, FastAPI, and PostgreSQL to provide a user-friendly and efficient platform. By using AI-based matching techniques, semantic similarity, and skill analysis, ResumeIQ helps candidates understand how well their resumes align with specific job roles.

The application identifies matching and missing skills, generates compatibility scores, and provides useful recommendations that help candidates recognize their strengths and areas requiring improvement.

The project was successfully deployed using Vercel for the frontend, Render for the backend, and Neon PostgreSQL for database management, making the application accessible through a public web link.

Overall, ResumeIQ demonstrates how Artificial Intelligence and full-stack development can be combined to simplify resume evaluation, improve career preparation, and help candidates make informed decisions about their professional development.

6.2 Future Scope

Although ResumeIQ provides essential resume analysis and job matching capabilities, several enhancements can be introduced in future versions to improve its functionality and effectiveness.

1. Advanced AI-Based Resume Analysis: Integrate more advanced AI models to improve resume understanding and job compatibility evaluation.

2. Automated Resume Improvement: Provide AI-generated suggestions to improve resume content, structure, keywords, and formatting based on specific job descriptions.

3. Personalized Learning Recommendations: Recommend courses, learning materials, and practical assignments based on the skills missing from a candidate's

resume.

4. Interview Preparation Module: Introduce job-role-based mock interviews, technical assessments, and interview question generation.

5. Advanced Skill Gap Analysis: Provide detailed skill-level assessments and identify technical and non-technical weaknesses.

6. Job Recommendation System: Recommend suitable job opportunities based on candidate skills, experience, qualifications, and career preferences.

7. Enhanced RAG Integration: Extend the Retrieval-Augmented Generation pipeline to provide more context-aware explanations and personalized recommendations.

8. Multiple Resume Format Support: Introduce support for additional document formats such as DOC and DOCX.

9. Resume Ranking and Comparison: Allow recruiters to compare multiple candidate resumes against a specific job description.

10. Performance and Scalability Improvements: Optimize application performance, database operations, and AI processing to support a larger number of users.

6.3 Final Outcome

ResumeIQ provides a centralized platform that bridges the gap between candidate qualifications and industry job requirements. By combining AI-powered analysis, skill-gap detection, and personalized recommendations, the system helps job seekers understand their current capabilities and identify opportunities for professional growth.

The project establishes a foundation for developing more intelligent career assistance platforms that support candidates throughout their job preparation journey.